



One Class Classifier with Vector Quantization

Working Loss Functions and Ideas

Daniel Staps & Ronny Schubert

Outline

Introduction

Loss Functions

What we can do and what we cannot do

Differences to other Fields

Outlier Detection:

- ▶ Detection of a small amount of datapoints which will be denoted as outliers

Classification Reject:

- ▶ Rejection of Datapoints laying in the margin of unsecurity for a classification task

Similaritys to other Fields

(Global) Reject Option

- ▶ One Class Classifiers can be seen as a Reject Model
- ▶ Aim to accept Datapoints for further processing which are in range of acceptable properties

Classifiers

- ▶ OCCs can be seen as a Classifier, but we represent just one Class

The Field of One Class Classifiers

- ▶ Density estimation methods
- ▶ Reconstruction methods
- ▶ Boundary methods e.g. Support Vector Data Description [24], One-class Support Vector Machine [25], Nearest Neighbor Data Description [1]
- ▶ Ensemble methods

1

¹Paper: A First Attempt on Evolutionary Prototype Reduction for Nearest Neighbor One-Class Classification

Other Boundary One Class Classifier Methods

- ▶ concentrates on enclosing the training data with a boundary
- ▶ aim to find optimal size of volume enclosing training data [23]
- ▶ to small boundary -> overtraining; to big boundary -> high acceptance of outliers
- ▶ can efficiently work with lower number training examples than other methods
- ▶ e.g. Support Vector Data Description [24], One-class Support Vector Machine [25], Nearest Neighbor Data Description [1]

Prerequisites

Given a training set $\mathcal{T} = \{(x_1, c_{x_1}), \dots, (x_n, c_{x_n})\}$ with $x_i \in \mathcal{X}$ being a data sample from the data set $\mathcal{X}$ and $c_{x_i} \in \{-1, 1\}$ the sample's corresponding label. Furthermore, given a prototype set $\mathcal{W}$, we define $c_\omega = 1$ for all $\omega \in \mathcal{W}$.

Thus, let ω^* be the best matching prototype to a given data sample x , such that

$$\omega^* = \arg \min_{\omega \in \mathcal{W}} d(x, \omega), \quad (1)$$

with $d(x, y)$ being the squared euclidean distance. Furthermore, a boundary parameter θ_ω is introduced for each ω .

For the sake of simplification and readability we write $x = x^+$ if $c_x = 1$ and $x = x^-$ if $c_x = -1$.

Heuristic Loss Function

Intuitively speaking, the classifier makes a wrong classification if $d(x^-, \omega^*) \leq \theta_{\omega^*}$ or if $d(x^+, \omega^*) > \theta_{\omega^*}$, thus the loss L_H is constructed as

$$L_H = \sum_{x^+} d(x^+, \omega^*) + \sum_{x^+} \beta_{\omega^*}^+ + \sum_{x^-} \beta_{\omega^*}^- \quad (2)$$

$$= (TP + FN) + Error_{boundary} , \quad (3)$$

with

$$\beta^{+/-} = \begin{cases} d(x^+, \omega^*) - \theta_{\omega^*} & \text{if } d(x^+, \omega^*) > \theta_{\omega^*} \\ \theta_{\omega^*} - d(x^-, \omega^*) & \text{if } d(x^-, \omega^*) < \theta_{\omega^*} \\ 0 & \text{otherwise} , \end{cases} \quad (4)$$

the error relative to the boundary θ and $d(x, y)$ again being the squared euclidean distance.

Heavy-tailed Solution

An idea to include the optimization of the data samples at the boundary θ was to consider a heavy-tailed distribution. The chosen distribution here is the student-t distribution $s(x)$ with one degree of freedom in location-scale version (aka Cauchy-distribution).

$$L_P = \sum_{x^+: d(x^+, \omega^*) > \theta_{\omega^*}} (1 - s(x^+)) + \sum_{x^-: d(x^-, \omega^*) \leq \theta_{\omega^*}} s(x^-) \quad (5)$$

$$= FN + FP, \quad (6)$$

with

$$S(x) = \frac{1}{\pi \theta_{\omega^*}} \left(1 + \frac{d(x, \omega^*)}{\theta_{\omega^*}^2} \right). \quad (7)$$

Inference Options

Assume $\tilde{x}$ is a new data sample not seen by the classifier during training

► Crisp

$$\tilde{x} = \begin{cases} \tilde{x}^+ & \text{if } d(\tilde{x}, \omega^*) \leq \theta_{\omega^*} \\ \tilde{x}^- & \text{if } d(\tilde{x}, \omega^*) > \theta_{\omega^*} . \end{cases} \quad (8)$$

► Soft $\rightarrow ??$

Properties of our One Class Classifier

Both methods work fine with:

- ▶ 1 or more Prototypes per Class (and multiple Classes) with the *Euclidean distance* in general
- ▶ when $\tilde{x}^+$ not fully surrounded by $\tilde{x}^-$, then ω^* and θ_{ω^*} may not result in the best solution
- ▶ L_P works well for OCC-LMVQ and L-OCC-MLVQ variants, though initialization can influence the result

What does not work now

- ▶ L_H does not work well for OCC-MLVQ and L-OCC-MLVQ, since it becomes instable (ω and θ act as resonance circuit)